

ML SEM: 2

P1

Perform following operations/functions based on arrays

a. Create a NumPy array of integers from 1 to 10.

b. Create a NumPy array of zeros of size 5.

c. Create a NumPy array of ones of shape (3, 3).

d. Find the sum and mean of a NumPy array.

e. Access the 3rd element from a NumPy array.

```
import numpy as np
```

a. Create a NumPy array of integers from 1 to 10

```
arr = np.arange(1, 11)
```

```
print("Array from 1 to 10:", arr)
```

b. Create a NumPy array of zeros of size 5

```
zeros_arr = np.zeros(5)
```

```
print("Zeros array of size 5:", zeros_arr)
```

c. Create a NumPy array of ones of shape (3, 3)

```
ones_arr = np.ones((3, 3))
```

```
print("Ones array of shape (3,3):")
```

```
print(ones_arr)
```

d. Find the sum and mean of a NumPy array

```
sample_arr = np.array([10, 20, 30, 40, 50])
```

```
print("Sample array:", sample_arr)
```

```
print("Sum of array:", np.sum(sample_arr))
```

```
print("Mean of array:", np.mean(sample_arr))
```

e. Access the 3rd element from a NumPy array

```
third_element = sample_arr[2] # index starts from 0
```

```
print("3rd element of the array:", third_element)
```

```
Array from 1 to 10: [ 1  2  3  4  5  6  7  8  9 10]
Zeros array of size 5: [0.  0.  0.  0.  0.]
Ones array of shape (3,3):
[[1.  1.  1.]
 [1.  1.  1.]
 [1.  1.  1.]]
Sample array: [10 20 30 40 50]
Sum of array: 150
Mean of array: 30.0
3rd element of the array: 30
```

P2

Perform following operations/functions based on arrays

a. Create a 2D array (3×4) and find its shape and dimension.

b. Find the maximum and minimum value from a NumPy array.

c. Perform element-wise addition and multiplication of two arrays.

d. Reshape a 1D array of size 12 into a 3×4 matrix.

e. Find all even numbers from a NumPy array using boolean indexing.

```
import numpy as np
```

a. Create a 2D array (3×4) and find its shape and dimension

```
arr_2d = np.array([
```

```
    [1, 2, 3, 4],
```

```
    [5, 6, 7, 8],
```

```
    [9, 10, 11, 12]
```

```
])
```

```
print("2D Array:")
```

```
print(arr_2d)
```

```
print("Shape of array:", arr_2d.shape)
```

```
print("Dimension of array:", arr_2d.ndim)
```

```
print()
```

b. Find the maximum and minimum value from a NumPy array

```

print("Maximum value:", np.max(arr_2d))
print("Minimum value:", np.min(arr_2d))
print()
# c. Element-wise addition and multiplication of two arrays
arr1 = np.array([1, 2, 3, 4])
arr2 = np.array([5, 6, 7, 8])
print("Array 1:", arr1)
print("Array 2:", arr2)
print("Element-wise Addition:", arr1 + arr2)
print("Element-wise Multiplication:", arr1 * arr2)
print()
# d. Reshape a 1D array of size 12 into a 3x4 matrix
arr_1d = np.arange(1, 13)
reshaped_arr = arr_1d.reshape(3, 4)
print("1D Array:", arr_1d)
print("Reshaped 3x4 Array:")
print(reshaped_arr)
print()
# e. Find all even numbers using boolean indexing
even_numbers = arr_1d[arr_1d % 2 == 0]
print("Even numbers from array:", even_numbers)

```

```

2D Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Shape of array: (3, 4)
Dimension of array: 2

Maximum value: 12
Minimum value: 1

Array 1: [1 2 3 4]
Array 2: [5 6 7 8]
Element-wise Addition: [ 6  8 10 12]
Element-wise Multiplication: [ 5 12 21 32]

1D Array: [ 1  2  3  4  5  6  7  8  9 10 11 12]
Reshaped 3x4 Array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

Even numbers from array: [ 2  4  6  8 10 12]

```

P3

Perform following operations based on NumPy – Array Creation & Properties

a. Create a NumPy array using arange() from 0 to 20 with step 2

b. Find the data type of a NumPy array

c. Change the data type of an array from int to float

d. Find the length of a NumPy array

```
import numpy as np
```

a. Create a NumPy array using arange() from 0 to 20 with step 2

```
arr = np.arange(0, 21, 2)
```

```
print("Array from 0 to 20 with step 2:", arr)
```

b. Find the data type of the NumPy array

```
print("Data type of array:", arr.dtype)
```

c. Change the data type of array from int to float

```
arr_float = arr.astype(float)
```

```
print("Array after changing data type to float:", arr_float)
```

```
print("New data type:", arr_float.dtype)
```

d. Find the length of the NumPy array

```
print("Length of the array:", len(arr))
```

```

Array from 0 to 20 with step 2: [ 0  2  4  6  8 10 12 14 16 18 20]
Data type of array: int32
Array after changing data type to float: [ 0.  2.  4.  6.  8. 10. 12. 14.
 16. 18. 20.]
New data type: float64
Length of the array: 11

```

P4

Perform following operations based on NumPy – Mathematical Operations

a. Find the square of each element in a NumPy array

b. Find the maximum element row-wise in a 2D array

c. Calculate the standard deviation of an array

d. Find the index of the maximum element in an array

```
import numpy as np
```

Create a sample NumPy array

```

arr = np.array([2, 4, 6, 8, 10])
print("Original Array:", arr)
# a. Find the square of each element in the array
square_arr = np.square(arr)
print("Square of each element:", square_arr)
print()
# Create a 2D array for row-wise operations
arr_2d = np.array([
    [10, 20, 30],
    [5, 15, 25],
    [7, 14, 21]
])
print("2D Array:")
print(arr_2d)
# b. Find the maximum element row-wise in a 2D array
row_max = np.max(arr_2d, axis=1)
print("Row-wise maximum elements:", row_max)
print()
# c. Calculate the standard deviation of an array
std_dev = np.std(arr)
print("Standard Deviation of array:", std_dev)
print()
# d. Find the index of the maximum element in an array
max_index = np.argmax(arr)
print("Index of maximum element:", max_index)

```

Original Array: [2 4 6 8 10]
Square of each element: [4 16 36 64 100]
2D Array:
[[10 20 30]
[5 15 25]
[7 14 21]]
Row-wise maximum elements: [30 25 21]
Standard Deviation of array: 2.8284271247461903
Index of maximum element: 4

P5

Perform following operations based on NumPy - Indexing & Slicing

a. Slice first five elements from an array

b. Access the last element of an array

c. Extract the second row from a 2D array

d. Reverse a NumPy array

```
import numpy as np
```

Create a sample 1D NumPy array

```
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])
```

```
print("Original Array:", arr)
```

a. Slice first five elements from the array

```
first_five = arr[:5]
```

```
print("First five elements:", first_five)
```

b. Access the last element of the array

```
last_element = arr[-1]
```

```
print("Last element:", last_element)
```

```
print()
```

Create a 2D NumPy array

```
arr_2d = np.array([
```

```
    [1, 2, 3],
```

```
    [4, 5, 6],
```

```
    [7, 8, 9]
```

```
])
```

```
print("2D Array:")
```

Original Array: [10 20 30 40 50 60 70 80]

First five elements: [10 20 30 40 50]

Last element: 80

2D Array:

```
[[1 2 3]
```

```
[4 5 6]
```

```
[7 8 9]]
```

Second row: [4 5 6]

Reversed array: [80 70 60 50 40 30 20 10]

```

print(arr_2d)
# c. Extract the second row from the 2D array
second_row = arr_2d[1]
print("Second row:", second_row)
# d. Reverse the NumPy array
reversed_arr = arr[::-1]
print("Reversed array:", reversed_arr)

```

P6

```

# Perform following operations based on NumPy – Random Module
# a. Generate 10 random integers between 1 and 100
# b. Create a 3×3 array of random numbers
# c. Set a random seed and generate random numbers
# d. Find the mean of randomly generated numbers
import numpy as np

# a. Generate 10 random integers between 1 and 100
random_integers = np.random.randint(1, 101, size=10)
print("10 Random integers between 1 and 100:", random_integers)

# b. Create a 3×3 array of random numbers (float values between 0 and 1)
random_3x3 = np.random.rand(3, 3)
print("\n3×3 Array of random numbers:")
print(random_3x3)

# c. Set a random seed and generate random numbers
np.random.seed(42) # set seed output are same for all time
seeded_random_numbers = np.random.rand(5)
print("\nRandom numbers after setting seed:", seeded_random_numbers)

# d. Find the mean of randomly generated numbers
mean_value = np.mean(seeded_random_numbers)
print("\nMean of randomly generated numbers:", mean_value)

```

10 Random integers between 1 and 100: [54 29 79 69 84 87 75 51 15 64]

3×3 Array of random numbers:
[[0.73192052 0.23746066 0.1053832]
 [0.23071968 0.42349495 0.06406116]
 [0.76389188 0.7363269 0.80982753]]

Random numbers after setting seed: [0.37454012 0.95071431 0.73199394 0.59865848 0.15601864]

Mean of randomly generated numbers: 0.5623850983416314

P7

```

# Attempt following questions based on Pandas – DataFrame Basics
# a. Create a DataFrame from a dictionary
# b. Display last 5 rows of a DataFrame
# c. Display column names of a DataFrame
# d. Find total number of rows and columns
import pandas as pd

# a. Create a DataFrame from a dictionary
data = {
    "Name": ["Amit", "Neha", "Rahul", "Priya", "Suresh", "Kiran", "Anita"],
    "Age": [23, 25, 22, 24, 26, 27, 21],
    "City": ["Delhi", "Mumbai", "Pune", "Delhi", "Chennai", "Bangalore", "Jaipur"]
}
df = pd.DataFrame(data)
print("DataFrame:")
print(df)
print()

# b. Display last 5 rows of the DataFrame
print("Last 5 rows of DataFrame:")

```

```

print(df.tail())
print()
# c. Display column names of the DataFrame
print("Column names:")
print(df.columns)
print()
# d. Find total number of rows and columns
rows, columns = df.shape
print("Total number of rows:", rows)
print("Total number of columns:", columns)

```

```

DataFrame:
   Name  Age  City
0  Amit  23  Delhi
1  Neha  25  Mumbai
2  Rahul  22   Pune
3  Priya  24   Delhi
4  Suresh 26  Chennai
5  Kiran  27  Bangalore
6  Anita  21   Jaipur

Last 5 rows of DataFrame:
   Name  Age  City
2  Rahul  22   Pune
3  Priya  24   Delhi
4  Suresh 26  Chennai
5  Kiran  27  Bangalore
6  Anita  21   Jaipur

Column names:
Index(['Name', 'Age', 'City'], dtype='object')

Total number of rows: 7
Total number of columns: 3

```

P8

```

# Attempt following questions based on Pandas
# a. Create a Pandas DataFrame with columns Roll_No, Name, and Marks for 5 students.
# b. Given a DataFrame df, write a statement to display the first 3 rows.
# c. From a DataFrame df, display only the Marks column
# d. Add a new column Result to a DataFrame where Result is "Pass" if Marks >= 40,
    else "Fail".
# e. Write a Pandas command to check for missing values in a DataFrame.

```

```
import pandas as pd
```

```

# a. Create a Pandas DataFrame for 5 students
data = {
    "Roll_No": [1, 2, 3, 4, 5],
    "Name": ["Amit", "Neha", "Rahul", "Priya", "Karan"],
    "Marks": [78, 35, 56, 42, 29]
}

```

```

df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
print()

```

```

# b. Display the first 3 rows of the DataFrame
print("First 3 rows of DataFrame:")
print(df.head(3))
print()

```

```

# c. Display only the Marks column
print("Marks column:")
print(df["Marks"])
print()

```

```

# d. Add a new column Result (Pass if Marks >= 40 else Fail)
df["Result"] = df["Marks"].apply(lambda x: "Pass" if x >= 40 else "Fail")
print("DataFrame after adding Result column:")
print(df)
print()

```

```

# e. Check for missing values in the DataFrame
print("Missing values in DataFrame:")
print(df.isnull())

```

```

Original DataFrame:
   Roll_No  Name  Marks
0         1  Amit    78
1         2  Neha    35
2         3  Rahul    56
3         4  Priya    42
4         5  Karan    29

```

```

First 3 rows of DataFrame:
   Roll_No  Name  Marks
0         1  Amit    78
1         2  Neha    35
2         3  Rahul    56

```

```

Marks column:
0    78
1    35
2    56
3    42
4    29
Name: Marks, dtype: int64

```

```

DataFrame after adding Result column:
   Roll_No  Name  Marks  Result
0         1  Amit    78     Pass
1         2  Neha    35     Fail
2         3  Rahul    56     Pass
3         4  Priya    42     Pass
4         5  Karan    29     Fail

```

```

Missing values in DataFrame:
   Roll_No  Name  Marks  Result
0  False  False  False  False
1  False  False  False  False
2  False  False  False  False
3  False  False  False  False
4  False  False  False  False

```

Attempt following questions based on Pandas

a. From a DataFrame containing student marks, select all students who scored more than 70 marks.

b. Sort the DataFrame in descending order of Marks.

c. Given a DataFrame with columns Department and Salary, find the average salary of each department.

d. Replace missing values in the Marks column with the average marks of the class.

e. Write Pandas code to read a CSV file named students.csv and display its basic information (rows, columns, data types).

```
import pandas as pd
import numpy as np

# Create a sample DataFrame of students
data = {
    "Roll_No": [1, 2, 3, 4, 5, 6],
    "Name": ["Amit", "Neha", "Rahul", "Priya", "Karan", "Sneha"],
    "Marks": [78, 65, 82, np.nan, 55, 91]
}

df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
print()

# a. Students scoring more than 70 marks
print("Students scoring more than 70 marks:")
print(df[df["Marks"] > 70])
print()

# b. Sort by Marks descending
print("Sorted DataFrame:")
print(df.sort_values(by="Marks", ascending=False))
print()

# c. Average salary by department
salary_data = {
    "Department": ["IT", "HR", "IT", "Finance", "HR", "Finance"],
    "Salary": [50000, 40000, 60000, 55000, 45000, 58000]
}

df_salary = pd.DataFrame(salary_data)
print("Average Salary by Department:")
print(df_salary.groupby("Department")["Salary"].mean())
print()

# d. Replace missing Marks with mean
avg_marks = df["Marks"].mean()
df["Marks"] = df["Marks"].fillna(avg_marks)
print("Updated DataFrame:")
print(df)
print()

# e. Read CSV and display info
df_csv = pd.read_csv("students_p9.csv")
print("CSV File Information:")
df_csv.info()
```

Original DataFrame:

	Roll_No	Name	Marks
0	1	Amit	78.0
1	2	Neha	65.0
2	3	Rahul	82.0
3	4	Priya	NaN
4	5	Karan	55.0
5	6	Sneha	91.0

Students scoring more than 70 marks:

	Roll_No	Name	Marks
0	1	Amit	78.0
2	3	Rahul	82.0
5	6	Sneha	91.0

Sorted DataFrame:

	Roll_No	Name	Marks
5	6	Sneha	91.0
2	3	Rahul	82.0
0	1	Amit	78.0
1	2	Neha	65.0
4	5	Karan	55.0
3	4	Priya	NaN

Average Salary by Department:

Department	Average Salary
Finance	56500.0
HR	42500.0
IT	55000.0

Name: Salary, dtype: float64

Updated DataFrame:

	Roll_No	Name	Marks
0	1	Amit	78.0
1	2	Neha	65.0
2	3	Rahul	82.0
3	4	Priya	74.2
4	5	Karan	55.0
5	6	Sneha	91.0

CSV File Information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Roll_No     10 non-null    int64
1   Name        10 non-null    object
2   Marks       9 non-null     float64
3   Department  10 non-null    object
dtypes: float64(1), int64(1), object(2)
memory usage: 452.0+ bytes
```


#students_p9.csv

```
Roll_No,Name,Marks,Department
1,Amit,78,IT
2,Neha,65,HR
3,Rahul,82,IT
4,Priya,,Finance
5,Karan,55,HR
6,Sneha,91,Finance
7,Ankit,68,IT
8,Meera,74,HR
9,Arjun,88,Finance
10,Simran,80,IT
```

P10

```
# Create a CSV file named students.csv containing the following columns:
# Roll_No, Name, Marks, Result. Write a Python program using the
# Pandas library to perform the following operations on the DataFrame:
# a. Load the data from the students.csv file into a Pandas DataFrame.
# b. Display all student records where the Marks obtained are greater than 60.
# c. Display the records of students whose Marks are between 50 and 80.
# d. Display all student records where the Result is "Fail".
```

```
import pandas as pd
# a. Load data from students.csv into a DataFrame
df = pd.read_csv("students_p10.csv")
print("Complete DataFrame:")
print(df)
print()
# b. Display records where Marks > 60
print("Students with Marks greater than 60:")
print(df[df["Marks"] > 60])
print()
# c. Display records where Marks are between 50 and 80
print("Students with Marks between 50 and 80:")
print(df[(df["Marks"] >= 50) & (df["Marks"] <= 80)])
print()
# d. Display records where Result is 'Fail'
print("Students with Result = Fail:")
print(df[df["Result"] == "Fail"])
```

#students_p10.csv

```
Roll_No,Name,Marks,Result
1,Amit,78,Pass
2,Neha,45,Fail
3,Rahul,62,Pass
4,Priya,55,Pass
5,Karan,38,Fail
6,Sneha,81,Pass
7,Anita,50,Pass
```

Complete DataFrame:

	Roll_No	Name	Marks	Result
0	1	Amit	78	Pass
1	2	Neha	45	Fail
2	3	Rahul	62	Pass
3	4	Priya	55	Pass
4	5	Karan	38	Fail
5	6	Sneha	81	Pass
6	7	Anita	50	Pass

Students with Marks greater than 60:

	Roll_No	Name	Marks	Result
0	1	Amit	78	Pass
2	3	Rahul	62	Pass
5	6	Sneha	81	Pass

Students with Marks between 50 and 80:

	Roll_No	Name	Marks	Result
0	1	Amit	78	Pass
2	3	Rahul	62	Pass
3	4	Priya	55	Pass
6	7	Anita	50	Pass

Students with Result = Fail:

	Roll_No	Name	Marks	Result
1	2	Neha	45	Fail
4	5	Karan	38	Fail

P11

Create a CSV file named `employees_p11.csv` containing the following columns:
`Emp_ID`, `Name`, `Department`, `Salary`. Write a Python program using the
Pandas library to perform the following tasks:

a. Read the data from the `employees.csv` file into a Pandas DataFrame.

b. Display all employee records belonging to the IT department.

c. Display all employee records whose Salary is not equal to 40,000.

```
import pandas as pd
```

```
# a. Read data from employees_p11.csv into a DataFrame
```

```
df = pd.read_csv("employees_p11.csv")
```

```
print("Complete Employee Data:")
```

```
print(df)
```

```
print()
```

```
# b. Display all employee records belonging to the IT department
```

```
print("Employees from IT Department:")
```

```
print(df[df["Department"] == "IT"])
```

```
print()
```

```
# c. Display all employee records whose Salary is not equal to 40000
```

```
print("Employees with Salary not equal to 40000:")
```

```
print(df[df["Salary"] != 40000])
```

Complete Employee Data:

	Emp_ID	Name	Department	Salary
0	101	Amit	IT	50000
1	102	Neha	HR	40000
2	103	Rahul	IT	60000
3	104	Priya	Finance	45000
4	105	Karan	IT	40000
5	106	Sneha	HR	38000

Employees from IT Department:

	Emp_ID	Name	Department	Salary
0	101	Amit	IT	50000
2	103	Rahul	IT	60000
4	105	Karan	IT	40000

Employees with Salary not equal to 40000:

	Emp_ID	Name	Department	Salary
0	101	Amit	IT	50000
2	103	Rahul	IT	60000
3	104	Priya	Finance	45000
5	106	Sneha	HR	38000

#employees_p11.csv

```
Emp_ID,Name,Department,Salary
```

```
101,Amit,IT,50000
```

```
102,Neha,HR,40000
```

```
103,Rahul,IT,60000
```

```
104,Priya,Finance,45000
```

```
105,Karan,IT,40000
```

```
106,Sneha,HR,38000
```

P12

A DataFrame `df` contains employee details with columns `Emp_ID`, `Name`, `Department`, and `Salary`. Write a Pandas program to display all employees whose salary is between 30,000 and 60,000.

```
import pandas as pd
```

```
# Sample employee DataFrame
```

```
data = {
```

```
    "Emp_ID": [101, 102, 103, 104, 105, 106],
```

```
    "Name": ["Amit", "Neha", "Rahul", "Priya", "Karan", "Sneha"],
```

```
    "Department": ["IT", "HR", "IT", "Finance", "IT", "HR"],
```

```
    "Salary": [25000, 40000, 55000, 60000, 45000, 70000]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print("Employee Data:")
```

```
print(df)
```

```
print()
```

```
# Display employees with salary between 30,000 and 60,000
```

Employee Data:

	Emp_ID	Name	Department	Salary
0	101	Amit	IT	25000
1	102	Neha	HR	40000
2	103	Rahul	IT	55000
3	104	Priya	Finance	60000
4	105	Karan	IT	45000
5	106	Sneha	HR	70000

Employees with Salary between 30,000 and 60,000:

	Emp_ID	Name	Department	Salary
1	102	Neha	HR	40000
2	103	Rahul	IT	55000
3	104	Priya	Finance	60000
4	105	Karan	IT	45000


```
print("Employees with Salary between 30,000 and 60,000:")
print(df[(df["Salary"] >= 30000) & (df["Salary"] <= 60000)])
```

P13

Given a DataFrame having Department and Salary columns, write a Pandas program to calculate:

a. Total salary

b. Average salary for each department.

```
import pandas as pd
```

Sample DataFrame

```
data = {
```

```
    "Department": ["IT", "HR", "IT", "Finance", "HR", "Finance"],
```

```
    "Salary": [50000, 40000, 60000, 55000, 45000, 58000]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print("Employee Salary Data:")
```

```
print(df)
```

```
print()
```

a. Calculate total salary

```
total_salary = df["Salary"].sum()
```

```
print("Total Salary:", total_salary)
```

```
print()
```

b. Calculate average salary for each department

```
avg_salary_dept = df.groupby("Department")["Salary"].mean()
```

```
print("Average Salary for each Department:")
```

```
print(avg_salary_dept)
```

Employee Salary Data:

	Department	Salary
0	IT	50000
1	HR	40000
2	IT	60000
3	Finance	55000
4	HR	45000
5	Finance	58000

Total Salary: 308000

Average Salary for each Department:

Department	Average Salary
Finance	56500.0
HR	42500.0
IT	55000.0

Name: Salary, dtype: float64

P14

Handling Missing Values Using Mean- A DataFrame contains a column Marks with some missing values. Replace all missing values with the mean of the Marks column and display the updated DataFrame.

```
import pandas as pd
```

```
import numpy as np
```

Sample DataFrame with missing values

```
data = {
```

```
    "Roll_No": [1, 2, 3, 4, 5],
```

```
    "Name": ["Amit", "Neha", "Rahul", "Priya", "Karan"],
```

```
    "Marks": [78, np.nan, 56, np.nan, 64]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print("Original DataFrame:")
```

```
print(df)
```

```
print()
```

Calculate mean of Marks column

```
mean_marks = df["Marks"].mean()
```

```
print("Mean of Marks:", mean_marks)
```

```
print()
```

Replace missing values with mean (Correct method)

```
df["Marks"] = df["Marks"].fillna(mean_marks)
```

Original DataFrame:

	Roll_No	Name	Marks
0	1	Amit	78.0
1	2	Neha	NaN
2	3	Rahul	56.0
3	4	Priya	NaN
4	5	Karan	64.0

Mean of Marks: 66.0

Updated DataFrame after replacing missing values:

	Roll_No	Name	Marks
0	1	Amit	78.0
1	2	Neha	66.0
2	3	Rahul	56.0
3	4	Priya	66.0
4	5	Karan	64.0

```
print("Updated DataFrame after replacing missing values:")
print(df)
```

P15

Remove Duplicate Records. A DataFrame contains duplicate student records. Write Pandas program to identify and remove duplicate rows.

```
import pandas as pd
# Sample DataFrame with duplicate records
data = {
    "Roll_No": [1, 2, 3, 2, 4, 1],
    "Name": ["Amit", "Neha", "Rahul", "Neha", "Priya", "Amit"],
    "Marks": [78, 65, 82, 65, 74, 78]
}
df = pd.DataFrame(data)
print("Original DataFrame:")
print(df)
print()
# Identify duplicate rows
print("Duplicate rows (True means duplicate):")
print(df.duplicated())
print()
# Display only duplicate records
print("Duplicate Records:")
print(df[df.duplicated()])
print()
# Remove duplicate rows
df_cleaned = df.drop_duplicates()
print("DataFrame after removing duplicate records:")
print(df_cleaned)
```

Original DataFrame:

	Roll_No	Name	Marks
0	1	Amit	78
1	2	Neha	65
2	3	Rahul	82
3	2	Neha	65
4	4	Priya	74
5	1	Amit	78

Duplicate rows (True means duplicate):

0	False
1	False
2	False
3	True
4	False
5	True

dtype: bool

Duplicate Records:

	Roll_No	Name	Marks
3	2	Neha	65
5	1	Amit	78

DataFrame after removing duplicate records:

	Roll_No	Name	Marks
0	1	Amit	78
1	2	Neha	65
2	3	Rahul	82
4	4	Priya	74

P16

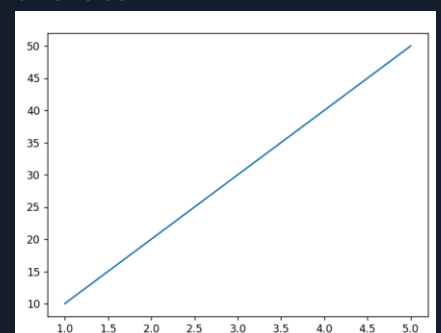
Attempt following questions based on matplotlib library

a. Write a program to plot a line graph showing the relationship between two lists of values.

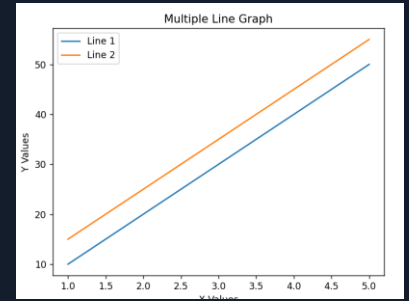
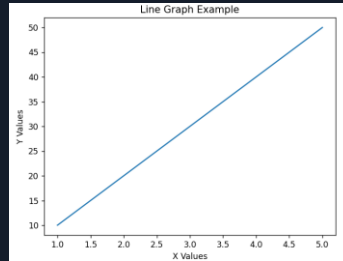
b. Write a program to draw a line graph and add: Title, X-axis label, Y-axis label

c. Write a Program to plot two or more lines on the same graph with labels.

```
import matplotlib.pyplot as plt
# Sample data
x = [1, 2, 3, 4, 5]
y1 = [10, 20, 30, 40, 50]
y2 = [15, 25, 35, 45, 55]
# a. Plot a simple line graph showing relationship between two lists
plt.plot(x, y1)
plt.show()
# b. Line graph with Title, X-axis label, and Y-axis label
plt.plot(x, y1)
plt.title("Line Graph Example")
plt.xlabel("X Values")
plt.ylabel("Y Values")
```



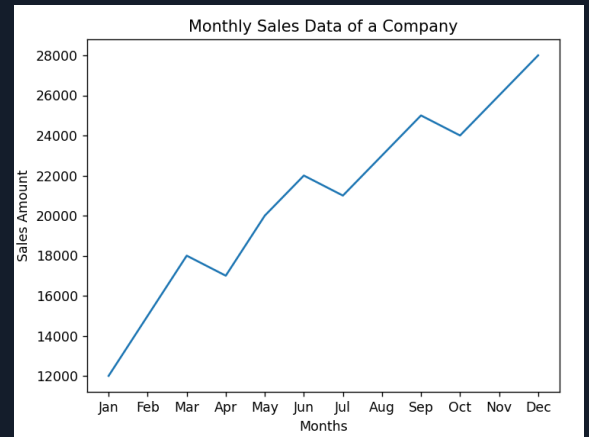
```
plt.show()
# c. Plot two lines on the same graph with labels
plt.plot(x, y1, label="Line 1")
plt.plot(x, y2, label="Line 2")
plt.title("Multiple Line Graph")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.legend()
plt.show()
```



P17

Plot a line graph showing monthly sales data of a company.

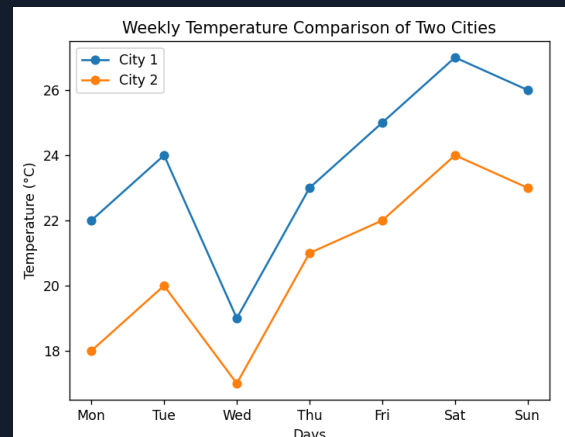
```
import matplotlib.pyplot as plt
# Monthly sales data
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun",
          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec"]
sales = [12000, 15000, 18000, 17000, 20000, 22000,
         21000, 23000, 25000, 24000, 26000, 28000]
# Plot line graph
plt.plot(months, sales)
plt.title("Monthly Sales Data of a Company")
plt.xlabel("Months")
plt.ylabel("Sales Amount")
plt.show()
```



P18

Draw a line graph to compare temperatures of two cities over a week.

```
import matplotlib.pyplot as plt
# Data for one week
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
city1_temps = [22, 24, 19, 23, 25, 27, 26]
city2_temps = [18, 20, 17, 21, 22, 24, 23]
# Plot line graph
plt.plot(days, city1_temps, marker='o', label="City 1")
plt.plot(days, city2_temps, marker='o', label="City 2")
# Add title and labels
plt.title("Weekly Temperature Comparison of Two Cities")
plt.xlabel("Days")
plt.ylabel("Temperature (°C)")
plt.legend()
# Display graph
plt.show()
```



P19

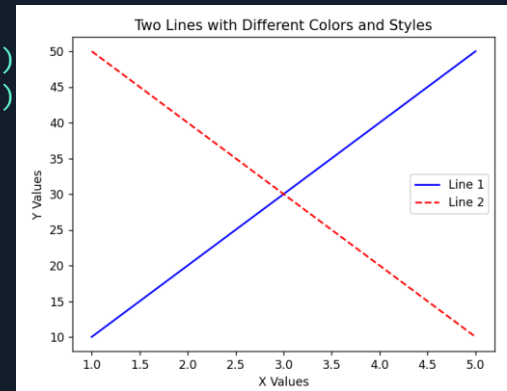
Plot two lines on the same graph with different colors and styles.

```
import matplotlib.pyplot as plt
# Sample data
x = [1, 2, 3, 4, 5]
y1 = [10, 20, 30, 40, 50]
```

```

y2 = [50, 40, 30, 20, 10]
# Plot two lines with different colors and styles
plt.plot(x, y1, color='blue', linestyle='-', label='Line 1')
plt.plot(x, y2, color='red', linestyle='--', label='Line 2')
# Add title and labels
plt.title("Two Lines with Different Colors and Styles")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.legend()
# Display graph
plt.show()

```



P20

Attempt following questions based on matplotlib library

a. Write a program to display a bar chart showing marks obtained by students in different subjects.

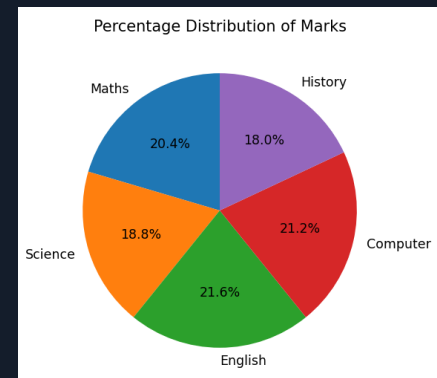
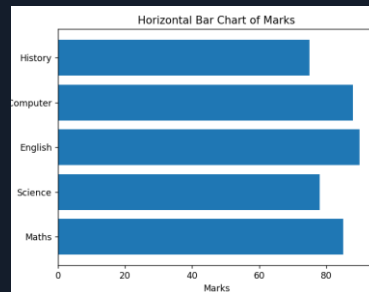
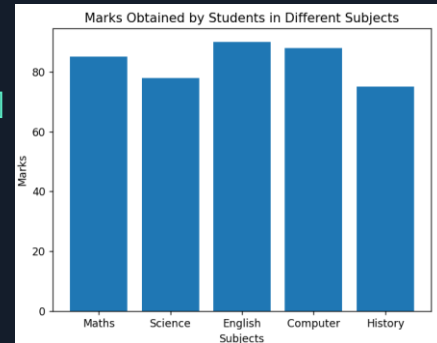
b. Write a program to draw a horizontal bar chart using Matplotlib.

c. Write a program to create a pie chart representing percentage distribution of data.

```

import matplotlib.pyplot as plt
# a. Bar chart - Marks in subjects
subjects = ["Maths", "Science", "English", "Computer", "History"]
marks = [85, 78, 90, 88, 75]
plt.bar(subjects, marks)
plt.title("Marks Obtained by Students in Different Subjects")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.show()
# b. Horizontal bar chart
plt.barh(subjects, marks)
plt.title("Horizontal Bar Chart of Marks")
plt.xlabel("Marks")
plt.ylabel("Subjects")
plt.show()
# c. Pie chart - Percentage distribution
labels = ["Maths", "Science", "English", "Computer", "History"]
sizes = [85, 78, 90, 88, 75]
plt.pie(sizes, labels=labels, autopct="%1.1f%%", startangle=90)
plt.title("Percentage Distribution of Marks")
plt.show()

```



P21

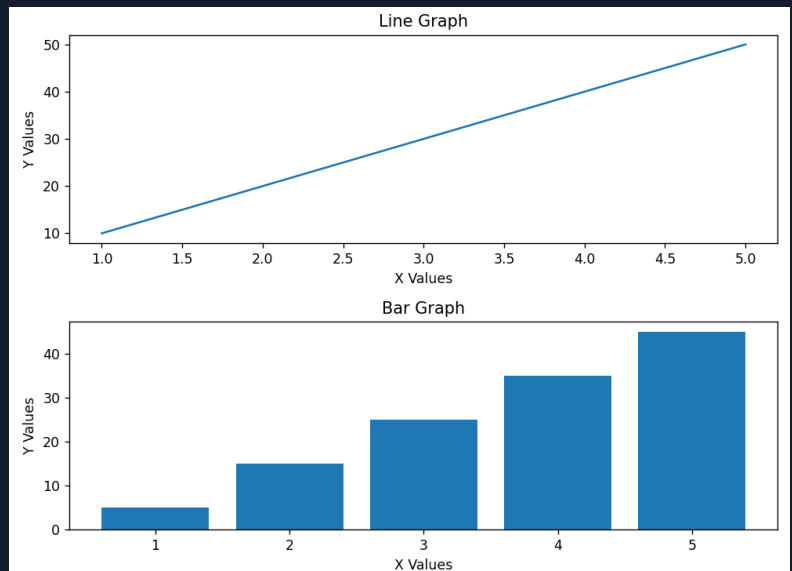
Plot two different graphs in the same figure.

```

import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y_line = [10, 20, 30, 40, 50]
y_bar = [5, 15, 25, 35, 45]
# Create a figure with 2 subplots
plt.figure(figsize=(8, 6))

```

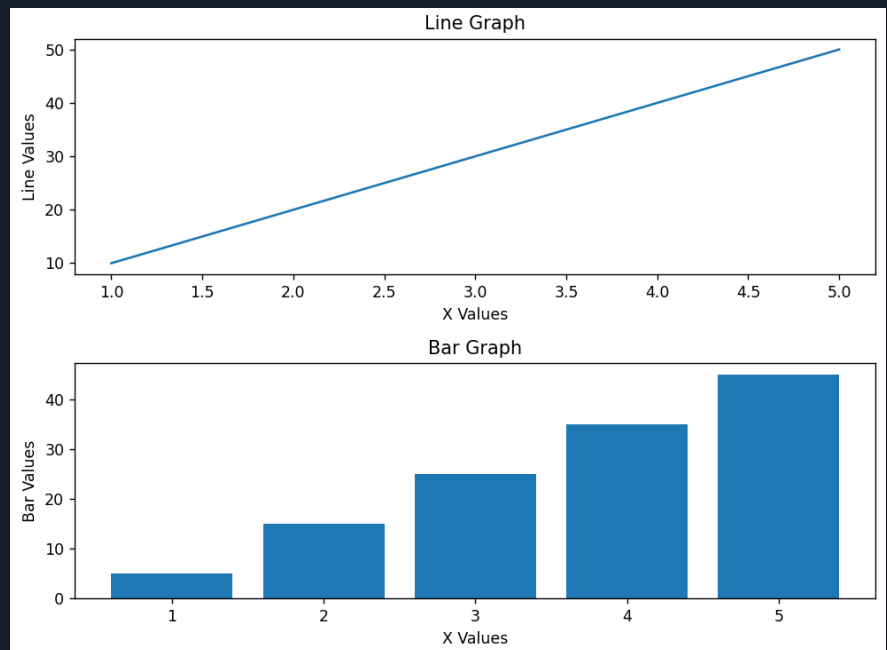
```
# First graph - Line graph
plt.subplot(2, 1, 1)
plt.plot(x, y_line)
plt.title("Line Graph")
plt.xlabel("X Values")
plt.ylabel("Y Values")
# Second graph - Bar graph
plt.subplot(2, 1, 2)
plt.bar(x, y_bar)
plt.title("Bar Graph")
plt.xlabel("X Values")
plt.ylabel("Y Values")
# Display the figure
plt.tight_layout()
plt.show()
```



P22

Plot line and bar graph together using subplots.

```
import matplotlib.pyplot as plt
# Sample data
x = [1, 2, 3, 4, 5]
line_values = [10, 20, 30, 40, 50]
bar_values = [5, 15, 25, 35, 45]
# Create figure
plt.figure(figsize=(8, 6))
# Line graph (subplot 1)
plt.subplot(2, 1, 1)
plt.plot(x, line_values)
plt.title("Line Graph")
plt.xlabel("X Values")
plt.ylabel("Line Values")
# Bar graph (subplot 2)
plt.subplot(2, 1, 2)
plt.bar(x, bar_values)
plt.title("Bar Graph")
plt.xlabel("X Values")
plt.ylabel("Bar Values")
# Adjust layout and display
plt.tight_layout()
plt.show()
```



P23

Write a Python program to detect outliers in a dataset.

```
import numpy as np
# Sample dataset
data = np.array([10, 12, 14, 15, 18, 20, 22, 25, 100])
print("Original Data:", data)
# Step 1: Calculate Q1 and Q3
Q1 = np.percentile(data, 25)
```

```

Q3 = np.percentile(data, 75)
# Step 2: Calculate IQR
IQR = Q3 - Q1
# Step 3: Define lower and upper bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
# Step 4: Detect outliers
outliers = data[(data < lower_bound) | (data > upper_bound)]
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
print("Outliers detected:", outliers)

```

Original Data: [10 12 14 15 18 20 22 25 100]
Lower Bound: 2.0
Upper Bound: 34.0
Outliers detected: [100]

P24

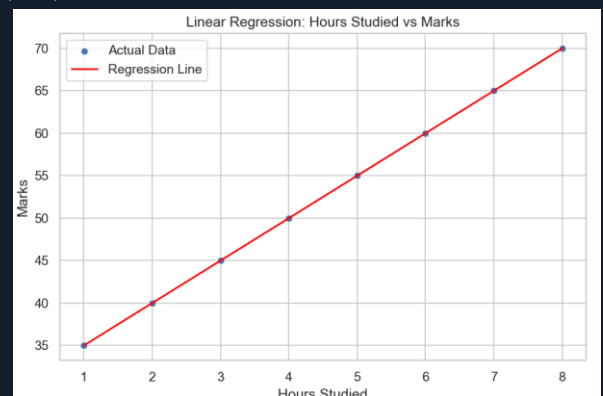
Write a Python program to apply Linear Regression using Scikit-learn and visualize the data and regression line using Seaborn

```

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
# Step 1: Create sample dataset
data = {
    "Hours_Studied": [1, 2, 3, 4, 5, 6, 7, 8],
    "Marks": [35, 40, 45, 50, 55, 60, 65, 70]
}
df = pd.DataFrame(data)
# Step 2: Prepare data for model
X = df[["Hours_Studied"]] # Independent variable (2D)
y = df["Marks"] # Dependent variable

# Step 3: Apply Linear Regression
model = LinearRegression()
model.fit(X, y)
# Predict values
df["Predicted_Marks"] = model.predict(X)
# Step 4: Visualize using Seaborn
sns.set(style="whitegrid")
plt.figure(figsize=(8, 5))
# Scatter plot of actual data
sns.scatterplot(x="Hours_Studied", y="Marks", data=df, label="Actual Data")
# Regression line
sns.lineplot(x="Hours_Studied", y="Predicted_Marks", data=df, color="red",
label="Regression Line")
plt.title("Linear Regression: Hours Studied vs Marks")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.legend()
plt.show()

```



P25

Write a Python program using Scikit-learn to classify students as Pass or Fail based on their marks using the K-Nearest Neighbors (KNN) algorithm.

```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Step 1: Create dataset
data = {
    "Marks": [25, 35, 45, 55, 65, 75, 85, 30, 50, 70],
    "Result": ["Fail", "Fail", "Pass", "Pass", "Pass", "Pass", "Pass", "Fail",
"Pass", "Pass"]
}
df = pd.DataFrame(data)
# Convert Result to numeric
df["Result"] = df["Result"].map({"Fail": 0, "Pass": 1})
# Step 2: Features and labels
X = df[["Marks"]]
y = df["Result"]
# Step 3: Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
# Step 4: Train KNN model
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
# Step 5: Predict test data
y_pred = knn.predict(X_test)
# Step 6: Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# Step 7: Predict new student result (Correct method)
new_marks = pd.DataFrame([[48]], columns=["Marks"])
prediction = knn.predict(new_marks)
if prediction[0] == 1:
    print("Student Result: Pass")
else:
    print("Student Result: Fail")
```

Classification Report:				
	precision	recall	f1-score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	2
accuracy			1.00	3
macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Student Result: Pass

P26

Write a Python program to implement the K-Nearest Neighbors (KNN) algorithm with K = 3 and classify a new data point using a simple dataset.

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
# Step 1: Create a simple dataset
# Feature: Marks
X = np.array([[35], [45], [55], [65], [75], [85]])
```

```

# Labels: 0 = Fail, 1 = Pass
y = np.array([0, 0, 1, 1, 1, 1])
# Step 2: Create KNN model with K = 3
knn = KNeighborsClassifier(n_neighbors=3)
# Train the model
knn.fit(X, y)
# Step 3: Classify a new data point
new_data = np.array([[50]]) # New student's marks
prediction = knn.predict(new_data)
# Step 4: Display result
if prediction[0] == 1:
    print("New Data Point Result: Pass")
else:
    print("New Data Point Result: Fail")

```

New Data Point Result: Fail

P27

```

# Change the value of K from 3 to 5 and observe the change in classification result.
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
# Step 1: Simple dataset
# Feature: Marks
X = np.array([[35], [45], [55], [65], [75], [85]])
# Labels: 0 = Fail, 1 = Pass
y = np.array([0, 0, 1, 1, 1, 1])
# New data point to classify
new_data = np.array([[50]])
# K = 3
knn_3 = KNeighborsClassifier(n_neighbors=3)
knn_3.fit(X, y)
result_k3 = knn_3.predict(new_data)
# K = 5
knn_5 = KNeighborsClassifier(n_neighbors=5)
knn_5.fit(X, y)
result_k5 = knn_5.predict(new_data)
# Display results
print("Result with K = 3:", "Pass" if result_k3[0] == 1 else "Fail")
print("Result with K = 5:", "Pass" if result_k5[0] == 1 else "Fail")

```

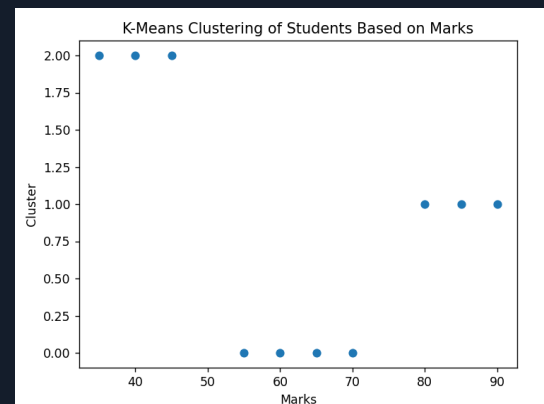
Result with K = 3: Fail
Result with K = 5: Pass

P28

```

# write a Python program to cluster students based on their marks using the K-Means clustering algorithm.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# Step 1: Create sample dataset
data = {
    "Marks": [35, 40, 45, 55, 60, 65, 70, 80, 85, 90]
}

```



```

df = pd.DataFrame(data)
print("Student Marks Data:")
print(df)
print()
# Step 2: Apply K-Means Clustering
# Number of clusters (e.g., Low, Medium, High)
kmeans = KMeans(n_clusters=3, random_state=42)
df["Cluster"] = kmeans.fit_predict(df[["Marks"]])
print("Data after K-Means Clustering:")
print(df)
print()
# Step 3: Visualize the clusters
plt.scatter(df["Marks"], df["Cluster"])
plt.title("K-Means Clustering of Students Based on Marks")
plt.xlabel("Marks")
plt.ylabel("Cluster")
plt.show()

```

Student Marks Data:

Marks

0	35
1	40
2	45
3	55
4	60
5	65
6	70
7	80
8	85
9	90

Data after K-Means Clustering:

Marks Cluster

0	35	2
1	40	2
2	45	2
3	55	0
4	60	0
5	65	0
6	70	0
7	80	1
8	85	1
9	90	1

